commuting conversions + join points in call-by-push-value

2026

Jonathan Chan
Univ. of Pennsylvania

Madi Gudin
Amherst College

Annabel Levy
UMBC

Stephanie Weirich
Univ. of Pennsylvania

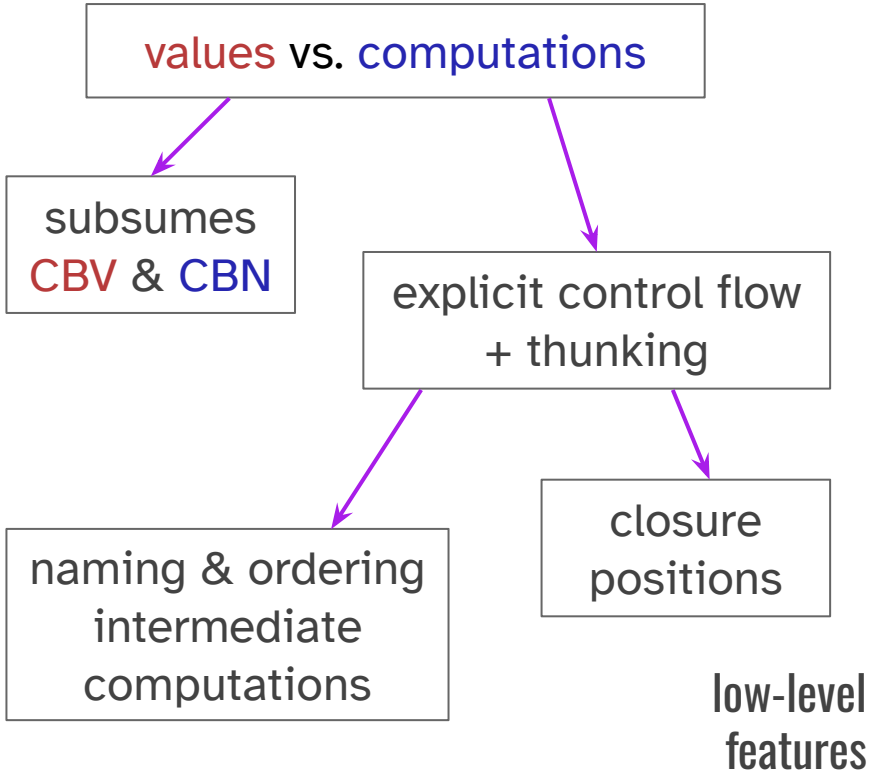
λ\REPL
penn-repl.github.io

the big picture:

CBPV as a compiler IR

† Paul B. Levy (2001, 2004)

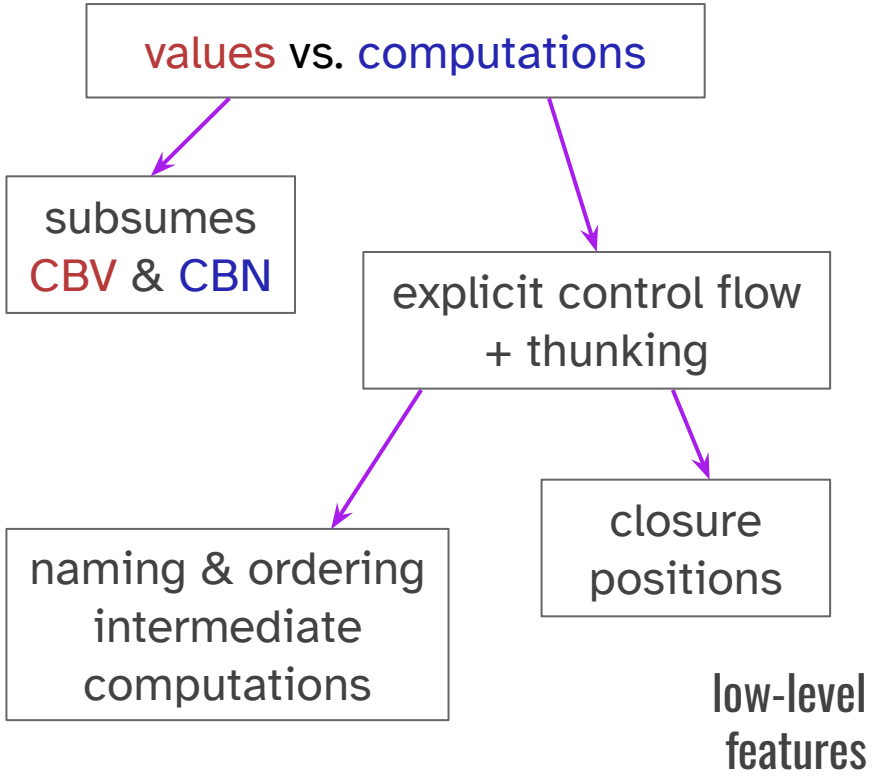
CBPV† as compiler IR



high-level features

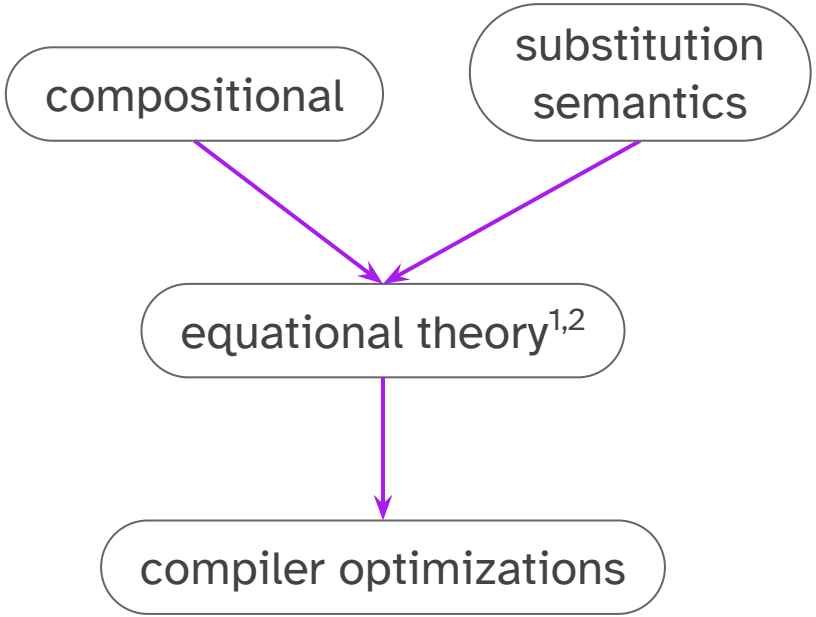
† Paul B. Levy (2001, 2004)

CBPV† as compiler IR



high-level features

[1] Rizkallah, Garbuzov, Zdancewic (2018)
[2] Forster, Schäfer, Spies, Stark (2019)



```
(λx. x + 3) ((λy. y + 1) 1)
```

call by value translation

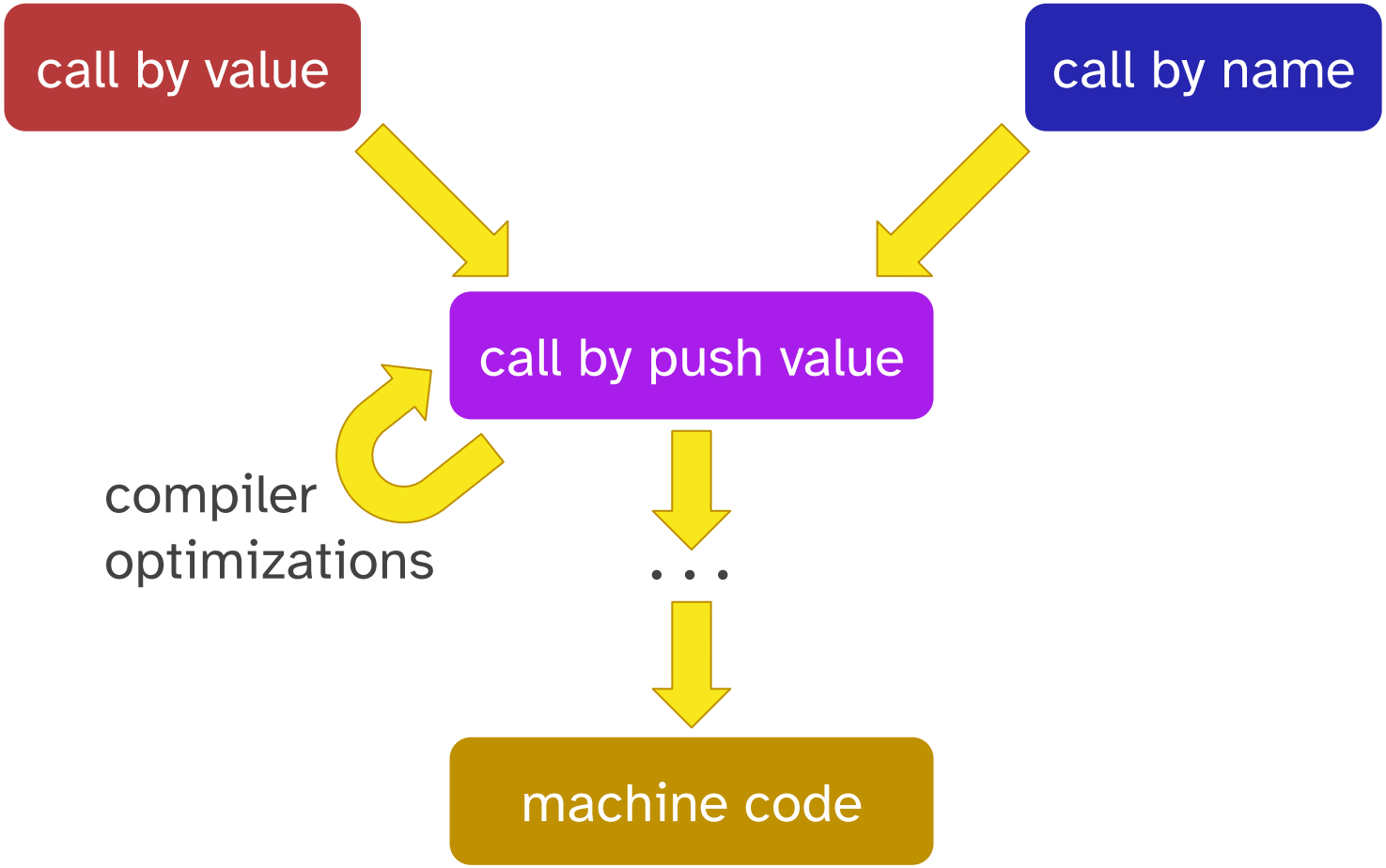
call by name translation

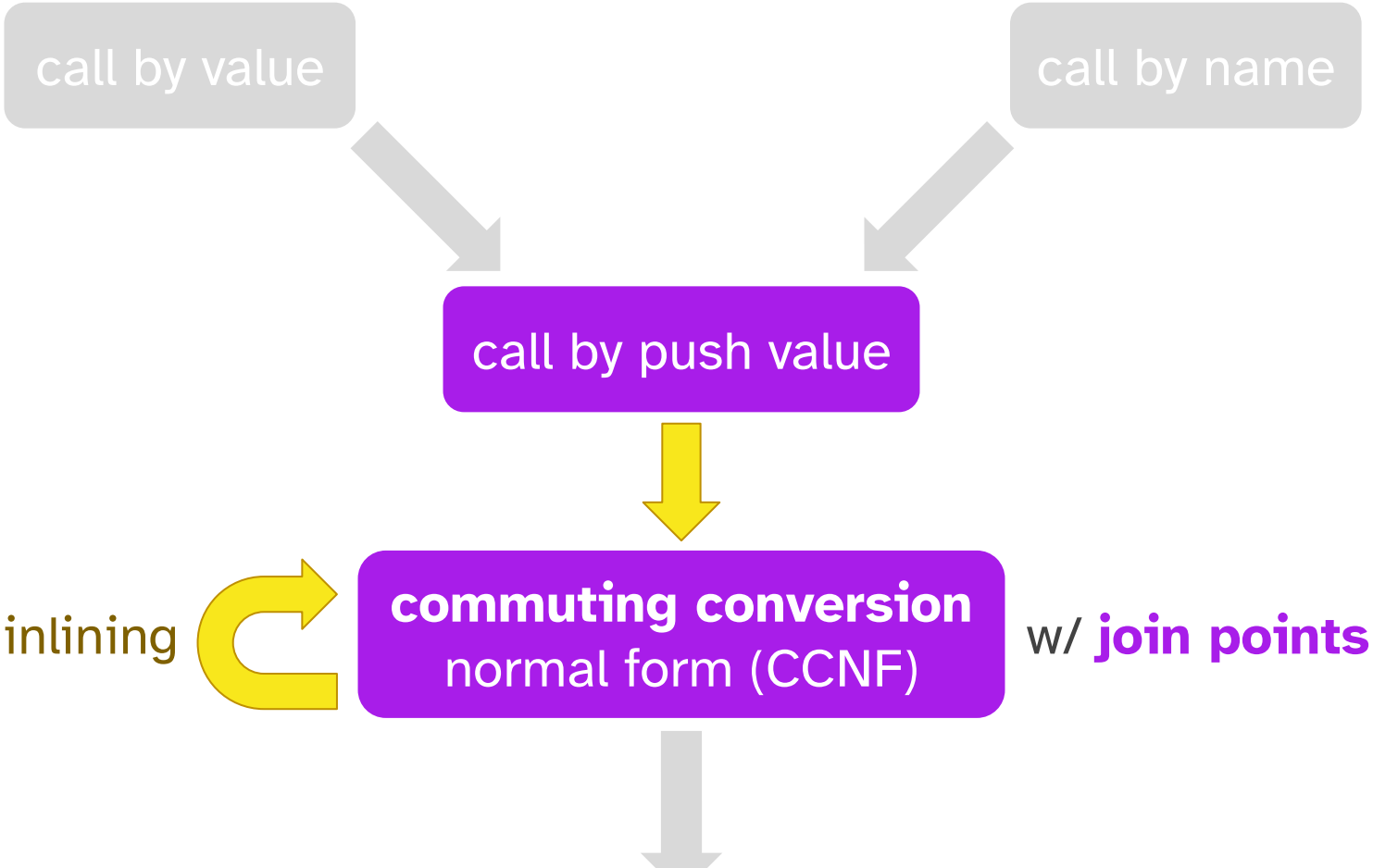
lambda calculus

call-by-push-value

```
let z ← (λy. return (y + 1)) 1
in (λx. return (x + 3)) z
```

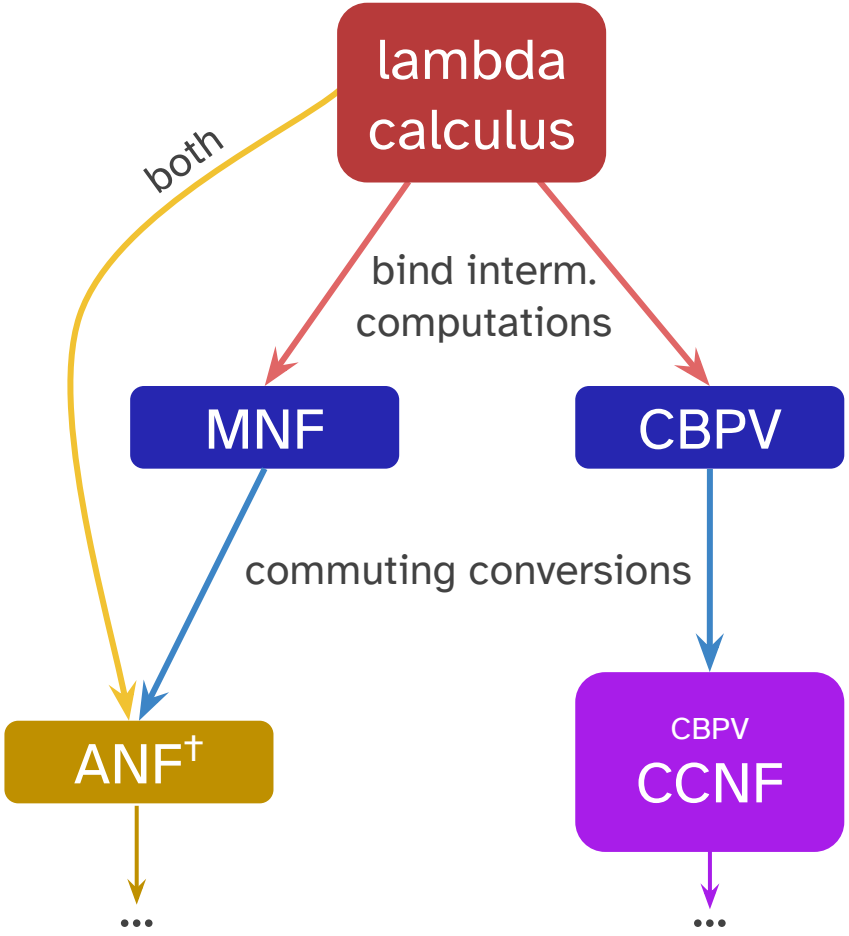
```
(λx. let x' ← x!
     in return (x' + 3))
{(λy. let y' ← y!
      in return (y' + 1))
 {ret 1}}
```


call by value
call by name
call by push value
compiler
optimizations
. . .
machine code


call by value
call by name
call by push value
inlining
commuting conversion
normal form (CCNF)
w/ join points

commuting conversion normal form

& **single-pass transformation** from CBPV into CCNF



†*The Essence of Compiling with Continuations*, PLDI 1993

CPBV + commuting conversions + join points

⓪ What CBPV is

① What commuting conversions are

② Why we need join points

③ When *inlining* comes in (β-reductions)

④ Comparison to GHC

call-by-push-value

Values

```
V,W ::= x | () | true | false | { M }
```

Comps

```
M,N ::= λx. M | M V
      |  return V | let x ← M in N
      |  if V then M else N
      |  V!
```

separate values and computations

separate positive intro forms from negative intro forms and eliminations

CBPV — computation rules

(no elimination rule for ())

$$\texttt{if true then } M_1 \texttt{ else } M_2 \rightsquigarrow M_1$$

$$\texttt{if false then } M_1 \texttt{ else } M_2 \rightsquigarrow M_2$$

$$\{M\}! \rightsquigarrow M$$

$$(\lambda x.\ M)\ V \rightsquigarrow M[x \mapsto V]$$

$$\texttt{let } x \leftarrow \texttt{return } V \texttt{ in } M \rightsquigarrow M[x \mapsto V]$$

❶ commuting conversions

```
          (let x ← N in   N' ) V
app-let ⇒ let x ← N in ((  N' ) V)
```

push negative elimination forms inside tail positions

❶ commuting conversions

```
          (let x ← N in   N' ) V
app-let ⇒ let x ← N in ((  N' ) V)

          let y ← (let x ← N in   N' ) in M
let-let ⇒ let x ← N in (let y ←   N'  in M)
```

push negative elimination forms inside tail positions

❶ commuting conversions + inlining

	(let x ← N in λy. M) V
app-let	⇒ let x ← N in ((λy. M) V)
β	⇒ let x ← N in M[y ↦ V]
	let y ← (let x ← N in return V) in M
let-let	⇒ let x ← N in (let y ← return V in M)
β	⇒ let x ← N in M[y ↦ V]

commuting conversions expose inlining opportunities

❷ commuting conversions

negative elimination forms (evaluation contexts)

$$E ::= \texttt{let}\ x \leftarrow \square\ \texttt{in}\ M \mid \square\ V$$

tail positions

$$L ::= \texttt{let}\ x \leftarrow N\ \texttt{in}\ \square \mid \texttt{if}\ V\ \texttt{then}\ \square\ \texttt{else}\ \square$$

commuting conversions

$$E[L[N]] \Rightarrow L[E[N]]$$

$$E[L[N_1, N_2]] \Rightarrow L[E[N_1], E[N_2]]$$

❷ commuting conversions

let-if

$$\begin{aligned} & \texttt{let } y \leftarrow (\texttt{if } V \texttt{ then } N_1 \texttt{ else } N_2) \texttt{ in } M \\ \Rightarrow\ & \texttt{if } V \texttt{ then } (\texttt{let } y \leftarrow N_1 \texttt{ in } M) \\ & \qquad \texttt{else } (\texttt{let } y \leftarrow N_2 \texttt{ in } M) \end{aligned}$$

app-if

$$\begin{aligned} & (\texttt{if } V \texttt{ then } N_1 \texttt{ else } N_2)\ W \\ \Rightarrow\ & (\texttt{if } V \texttt{ then } N_1\ W \texttt{ else } N_2\ W) \end{aligned}$$

❷ commuting conversions

let-if

$$\begin{aligned} & \texttt{let } y \leftarrow (\texttt{if } V \texttt{ then } N_1 \texttt{ else } N_2) \texttt{ in } M \\ \Rightarrow\ & \texttt{if } V \texttt{ then } (\texttt{let } y \leftarrow N_1 \texttt{ in } M) \\ & \qquad \texttt{else } (\texttt{let } y \leftarrow N_2 \texttt{ in } M) \end{aligned}$$

app-if

$$\begin{aligned} & (\texttt{if } V \texttt{ then } N_1 \texttt{ else } N_2)\ W \\ & \cancel{\Rightarrow\ (\texttt{if } V \texttt{ then } N_1\ W \texttt{ else } N_2\ W)} \\ \Rightarrow\ & \texttt{let } x \leftarrow \texttt{return } W \texttt{ in} \\ & (\texttt{if } V \texttt{ then } N_1\ x \texttt{ else } N_2\ x) \end{aligned}$$

❷ commuting conversions + join points

let-if

```
   let y ← (if v then N₁ else N₂) in M
⇒ if v then (let y ← N₁ in M)
            else (let y ← N₂ in M)
⇒ let f ← return {λy. M} in
  if v then (let y ← N₁ in f! y)
            else (let y ← N₂ in f! y)
```

❷ commuting conversions + join points

let-if

```
   let y ← (if v then N₁ else N₂) in M
⇒ if v then (let y ← N₁ in M)
            else (let y ← N₂ in M)
⇒ let f ← return {λy. M} in
  if v then (let y ← N₁ in f! y)
       else (let y ← N₂ in f! y)
```

- ☐ incurs cost of creating closure
- ☐ function calls may be expensive

❷ commuting conversions + primitive join points

let-if

```
  let y ← (if v then N₁ else N₂) in M
⇒ if v then (let y ← N₁ in m)
            else (let y ← N₂ in m)
⇒ join j y = M in
  if v then (let y ← N₁ in jump j y)
       else (let y ← N₂ in jump j y)
```

conditions:

- ☐ no jumps inside thunks
- ☐ jumps only in tail position

Compiling without Continuations, PLDI 2017

❸ commuting conversions in little steps

$$(\mathtt{let}\ x \leftarrow N_1\ \mathtt{in}\ (\lambda w.\ (\mathtt{let}\ y \leftarrow N_2\ \mathtt{in}\ (\lambda z.\ M))\ w))\ V$$

app-let

$$\Rightarrow \mathtt{let}\ x \leftarrow N_1\ \mathtt{in}\ ((\lambda w.\ (\mathtt{let}\ y \leftarrow N_2\ \mathtt{in}\ (\lambda z.\ M))\ w)\ V)$$

β

$$\Rightarrow \mathtt{let}\ x \leftarrow N_1\ \mathtt{in}\ ((\mathtt{let}\ y \leftarrow N_2\ \mathtt{in}\ (\lambda z.\ M))\ V)$$

app-let

$$\Rightarrow \mathtt{let}\ x \leftarrow N_1\ \mathtt{in}\ \mathtt{let}\ y \leftarrow N_2\ \mathtt{in}\ ((\lambda z.\ M)\ V)$$

β

$$\Rightarrow \mathtt{let}\ x \leftarrow N_1\ \mathtt{in}\ \mathtt{let}\ y \leftarrow N_2\ \mathtt{in}\ M[z \mapsto V]$$

❸ commuting conversions all at once

	$(\mathtt{let}\ x \leftarrow N_1\ \mathtt{in}\ (\lambda w.\ (\mathtt{let}\ y \leftarrow N_2\ \mathtt{in}\ (\lambda z.\ M))\ w))\ V$
app-let*	$\Rightarrow \mathtt{let}\ x \leftarrow N_1\ \mathtt{in}\ ((\lambda w.\ \mathtt{let}\ y \leftarrow N_2\ \mathtt{in}\ ((\lambda z.\ M)\ w))\ V)$
β	$\Rightarrow \mathtt{let}\ x \leftarrow N_1\ \mathtt{in}\ \mathtt{let}\ y \leftarrow N_2\ \mathtt{in}\ (\lambda z.\ M)\ v$
β	$\Rightarrow \mathtt{let}\ x \leftarrow N_1\ \mathtt{in}\ \mathtt{let}\ y \leftarrow N_2\ \mathtt{in}\ M[z \mapsto v]$

single-pass commuting conversion normalization

to CCNF subset of CBPV + join points

$[\![\cdot]\!]K$ $[\![\cdot]\!]$ *CC-normalization*

$\Gamma \mid \Delta \vdash M : B$ $\Gamma \vdash V : A$ *typing*

$M \rightsquigarrow^* M'$ *evaluation*

single-pass commuting conversion normalization

to CCNF subset of CBPV + join points

$[\![\cdot]\!]K$ $[\![\cdot]\!]$ *CC-normalization*

$\Gamma \mid \Delta \vdash M : B$ $\Gamma \vdash V : A$ *typing*

$M \rightsquigarrow^* M'$ *evaluation*

$\Gamma \mid \Delta \vDash M_1 \sim M_2 : B$ *semantic equivalence*

<u>Fundamental lemma</u>:

If $\Gamma \mid \cdot \vdash M : B$ then $\Gamma \mid \cdot \vDash M \sim [\![M]\!]\square : B$.

<u>Equivalence of CC-normalization</u>:

If $\cdot \mid \cdot \vdash M : F\ \mathtt{Bool}$ then $M \rightsquigarrow^* \mathtt{ret}\ V \ {}^*\!\!\leftsquigarrow [\![M]\!]\square$.

metatheory

mechanized in Lean 4

Related work: GHC Core and join points

- Larger language: polymorphism, datatypes, and recursion
- Step-by-step rewrites instead of normalization algorithm
  - More complexity in type system to check intermediate forms
  - ..but more flexibility in rewrites
- More evaluation contexts (i.e. *more* commuting conversions)
  - join point definitions (straightforward)
  - case-of-case (mixed)

④ λ-calculus case-of-case (pairs)

λ-calculus commuting conversion

```
case (case v of (x1,y1) ⇒ e1) of (x2,y2) ⇒ e2
⇒ case v of (x1,y1) ⇒ case e1 of (x2,y2) ⇒ e2
```

in CBPV

```
let z ← (case v of (x1,y1) ⇒ M1) in
case z of (x2,y2) ⇒ M2
```

let-case

```
⇒ case v of (x1,y1) ⇒ let z ← M1 in
case z of (x2,y2) ⇒ M2
```

④ λ-calculus case-of-case (if)

commuting conversion

```
if (if v then e2 else e3) then e4 else e5
⇒ if v then (if e2 then e4 else e5)
            else (if e3 then e4 else e5)
```

with join points

```
if (if v then e2 else e3) then e4 else e5
⇒
let j4 = e4 in let j5 = e5 in
if v then (if e2 then j4 else j5)
      else (if e3 then j4 else j5)
```

④ CBPV case-of-case (if)

λ

```
if (if v then e2 else e3) then e4 else e5
```

CBPV

```
let y ← (if v then M2 else M3) in
if y then M4 else M5
```

let-if

```
⇒ if v then (let y ← M2 if y then M4 else M5)
        else (let y ← M3 if y then M4 else M5)
```

④ CBPV case-of-case (if - join points)

λ

```
if (if v then e2 else e3) then e4 else e5
```

CBPV

```
let y ← (if v then M2 else M3) in
if y then M4 else M5
```

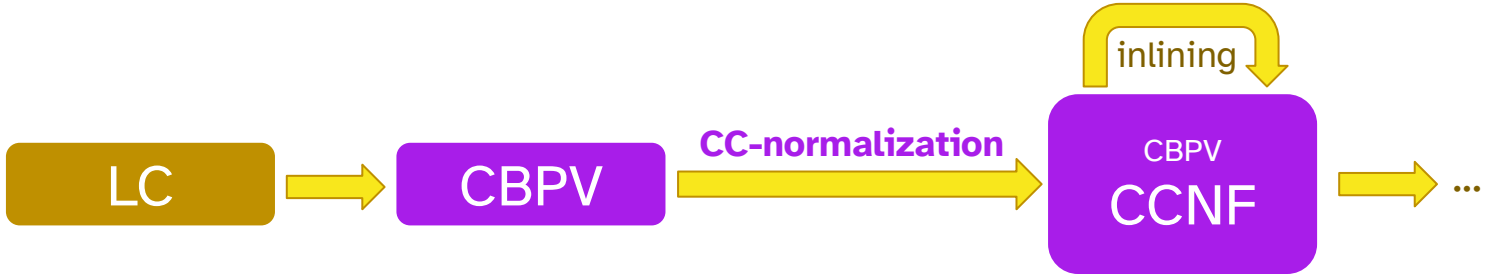
CCNF

```
⇒ join j y = if y then M4 else M5 in
  if v then (let y ← M2 in jump j y)
       else (let y ← M3 in jump j y)
```

④ CBPV case-of-case - alternative join points

```
join j y = if y then M4 else M5 in
if v then (let z ← M2 in jump j z)
     else (let z ← M3 in jump j z)
```

```
join j4 = M4 in join j5 = M5 in
if v then (let y ← M2 in if y then jump j4 else jump j5)
     else (let y ← M3 in if y then jump j4 else jump j5)
```



commuting conversions + join points in CBPV

paper draft: https://ionathan.ch/assets/pdfs/ccnf.pdf
mechanization: https://github.com/ionathanch/CBPV/tree/join